

The Evolution of AI Agents and Modern AI Engineering

A Technical Research Report | July 2026

1. LLM Applications

Large Language Models (LLMs) began as text-completion engines but rapidly evolved into the foundation of a new application layer. Early LLM applications were largely **single-turn, stateless** systems: a prompt goes in, text comes out. This pattern powered chatbots, summarizers, and copilots (e.g., GitHub Copilot's inline code suggestions, and early ChatGPT). As models improved in reasoning and instruction-following, developers began to chain LLM calls together, retrieve external knowledge (Retrieval-Augmented Generation, or RAG), and connect models to external systems — the precursor to what is now called **AI Engineering**: the discipline of designing, orchestrating, evaluating, and productionizing LLM-based systems, distinct from classical ML engineering because it centers on prompting, context management, and system composition rather than model training.

2. AI Agents

An **AI agent** is an LLM-driven system that autonomously pursues a goal by iteratively perceiving its environment, reasoning about next steps, taking actions (often via tools), and observing results — rather than producing one static response. This "agentic loop" is the defining architectural shift from LLM applications to agents.

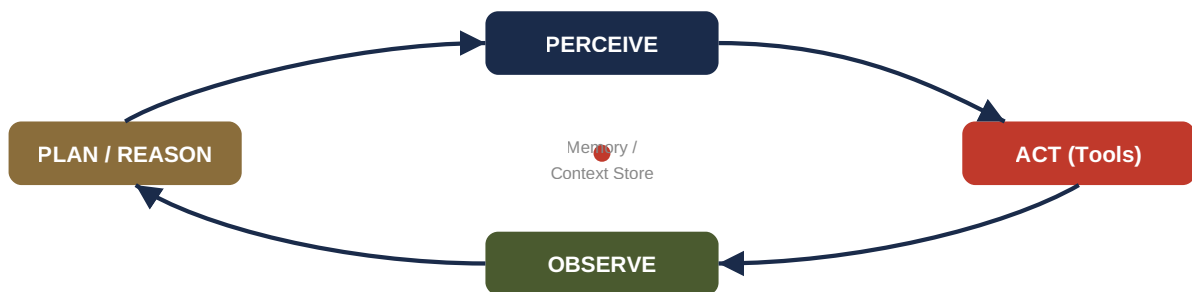


Figure 1 — The core agent loop: perceive → plan → act → observe, wrapped around persistent memory.

REAL-WORLD EXAMPLE: Anthropic's *Claude Code* and Cursor's *Agent Mode* perceive a codebase, plan a multi-file change, call file-editing and shell tools, and observe test output to self-correct — repeating until the goal is met.

3. Tool Calling

Tool calling (a.k.a. function calling) lets an LLM invoke external functions — APIs, code execution, databases, search — by emitting a structured request that a runtime executes and feeds back as an observation. This is what turns a text generator into an agent capable of acting on the world. Anthropic's **Model Context Protocol (MCP)** and OpenAI's function-calling API standardized this pattern, decoupling tool definitions from any single model provider.

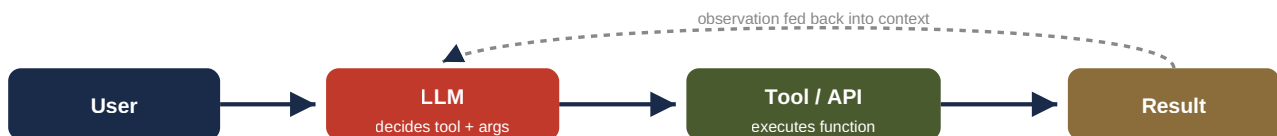


Figure 2 — Tool-calling loop: the model outputs a structured call, the runtime executes it, and the result re-enters context.

REAL-WORLD EXAMPLE: Perplexity and ChatGPT's browsing mode call a *search* tool mid-generation; Devin (Cognition Labs) calls shell, browser, and editor tools to autonomously resolve software tickets.

4. Memory

Agents need memory beyond a single context window. **Short-term (working) memory** lives in the prompt/context for the current task. **Long-term memory** persists across sessions, typically via vector databases (semantic recall), structured logs (episodic memory), or summarized "reflections" written back by the agent itself. Effective memory design balances recall quality against context-window cost and latency — over-stuffing context degrades reasoning, so agents increasingly use retrieval-on-demand rather than always-loaded memory.

REAL-WORLD EXAMPLE: ChatGPT's persistent "Memory" feature stores user facts across chats; Claude's project-level memory and Claude.ai's "reference past chats" retrieve relevant history rather than reloading everything.

5. Planning

Planning is how an agent decomposes a goal into an ordered sequence of actions. Key techniques include **Chain-of-Thought** (explicit intermediate reasoning), **ReAct** (interleaving reasoning traces with actions, Yao et al., 2022), **Tree-of-Thought** (exploring multiple reasoning branches), and **Reflexion**-style self-critique, where an agent evaluates its own output and re-plans on failure. Modern frameworks combine these with explicit task graphs (e.g., a planner LLM producing a DAG of subtasks for a worker to execute).

REAL-WORLD EXAMPLE: AutoGPT and BabyAGI popularized fully autonomous plan-execute-reflect loops; Claude Code plans a todo list before editing code and updates it as tasks complete.

6. Multi-Agent Systems

Rather than one monolithic agent handling every subtask, **multi-agent systems** assign specialized roles — e.g., an orchestrator that decomposes work and delegates to worker agents (researcher, coder, reviewer), each with its own tools, context, and sometimes its own model. This mirrors human team structures and improves reliability by isolating context and responsibility per agent, at the cost of added coordination complexity and token overhead.

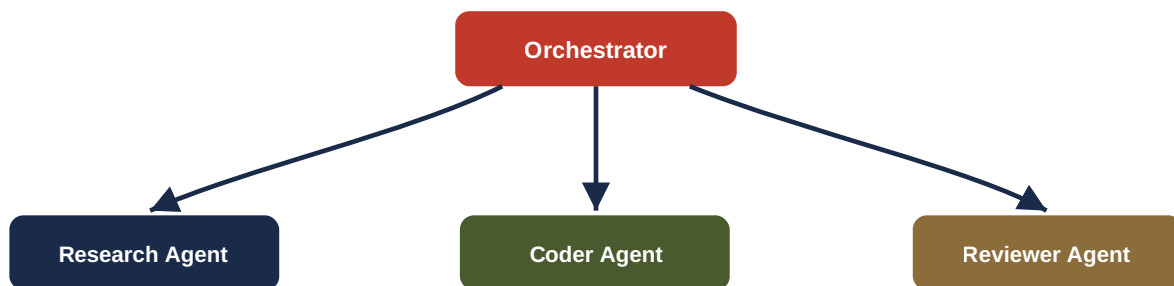


Figure 3 — Orchestrator-worker topology: a lead agent delegates subtasks to specialized sub-agents and aggregates results.

REAL-WORLD EXAMPLE: Anthropic's multi-agent Research feature runs an orchestrator that spawns parallel subagents to explore different query angles; Microsoft's AutoGen and CrewAI provide open frameworks for orchestrator/worker and debate-style multi-agent pipelines.

7. Future of AI Engineering

AI engineering is shifting from prompt-crafting toward **systems design**: standardized tool protocols (MCP), agent evaluation harnesses, context-window economics, and guardrails for autonomy. Expect greater emphasis on: (1) *interoperability* — agents and tools speaking common protocols across vendors; (2) *evaluation-driven development* — agents tested against task benchmarks before deployment, not just prompts reviewed by eye; (3) *hybrid human-agent workflows* with explicit checkpoints for high-stakes actions; and (4) *cost/latency-aware orchestration*, routing subtasks to smaller or larger models as needed. The discipline is converging with classical software engineering practice — version control, CI/CD, and observability — applied to non-deterministic, language-driven systems.

References

- [1] Yao, S. et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629.
- [2] Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366.
- [3] Schick, T. et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv:2302.04761.
- [4] Anthropic (2024–2026). *Model Context Protocol Specification*. modelcontextprotocol.io
- [5] Anthropic (2025). *Building Effective Agents*. anthropic.com/research/building-effective-agents
- [6] OpenAI (2023). *Function calling and other API updates*. openai.com/blog
- [7] Wu, Q. et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155.
- [8] Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401.

Prepared as a technical research overview · diagrams are original schematics summarizing publicly documented agent architectures.